

I am using a React frontend with Redux Toolkit + RTK Query and a backend API with versioned `/v1` routes.

I have decided to use a **cookie-based authentication architecture**.

PRIMARY AUTH ARCHITECTURE

Use this model consistently across the entire frontend:

```
Redux:
  accessToken
  user

Browser HttpOnly Cookie:
  refreshToken
```

Important security rules

1. The refresh token MUST NOT be stored in:
2. Redux
3. localStorage
4. sessionStorage
5. JavaScript-accessible cookies
6. The refresh token is managed entirely by the backend using an `HttpOnly` cookie.
7. Frontend requests must use:

```
credentials: "include"
```

so the browser sends the refresh cookie automatically.

1. The access token may exist in Redux memory.
2. Do NOT persist the access token or refresh token to localStorage unless there is an existing non-authentication reason to do so. Prefer memory-only access-token storage.
3. Do not expose the refresh token to JavaScript.

FILES / AREAS TO FIX

Review and fix the entire authentication flow, including:

- `baseApi`
- `authSlice`
- `authApi`
- `authSession`
- AuthProvider / session bootstrap
- profile API
- logout flow
- RTK Query cache handling
- login
- registration
- refresh
- OAuth login
- application startup
- concurrent 401 requests
- final 401 handling
- CSRF handling

Do not make isolated changes that create inconsistencies between these files.

Trace the complete lifecycle before modifying code.

1. AUTH SLICE

The Redux auth state should contain only:

```
{
  accessToken: null,
  user: null
}
```

Remove:

```
refreshToken
```

from Redux.

The refresh token belongs exclusively to the backend HttpOnly cookie.

Use clear, predictable reducers.

Recommended structure:

```
const initialState = {  
  accessToken: null,  
  user: null,  
};
```

Use:

```
setSession
```

for replacing the complete authenticated session:

```
setSession: (state, action) => {  
  state.accessToken = action.payload?.accessToken ?? null;  
  state.user = action.payload?.user ?? null;  
}
```

Use a separate action such as:

```
setAccessToken
```

when only the access token changes after refresh.

Use:

```
clearCredentials
```

to completely clear authentication.

Do not use nullish-merging behavior such as:

```
accessToken ?? state.accessToken
```

for session replacement because that can accidentally retain stale credentials.

There must never be a situation where an old access token survives a logout or a new session.

2. REMOVE REFRESH TOKEN PERSISTENCE

Find every place that reads, writes, serializes, or persists:

```
refreshToken
```

Remove that frontend persistence logic.

In particular, eliminate logic equivalent to:

```
safeLocalStorageSet(  
  SESSION_KEY,  
  JSON.stringify({  
    accessToken,  
    refreshToken,  
    user  
  })  
)
```

Do not persist refresh tokens.

If an existing session-storage/local-storage helper is being used for auth credentials, refactor it so it does not contain the refresh token.

Prefer:

```
HttpOnly refresh cookie  
+  
Redux memory access token
```

3. RTK QUERY BASE API

Keep:

```
credentials: "include"
```

on the base query.

The normal authenticated request should work like this:

```
Redux accessToken
  ↓
Authorization: Bearer <accessToken>
  ↓
API request
```

The refresh request should work like this:

```
POST /auth/refresh
  ↓
credentials: include
  ↓
Browser automatically sends HttpOnly refresh cookie
  ↓
Backend validates refresh cookie
  ↓
Backend rotates refresh cookie if applicable
  ↓
Backend returns new access token
```

The frontend must NOT manually send a refresh token.

4. SEPARATE REFRESH BASE QUERY

Keep a separate public base query for refresh.

For example:

```
const publicBaseQuery = fetchBaseQuery({
  baseUrl: apiBaseUrl,
  credentials: "include",
  prepareHeaders: prepareCookieHeaders,
});
```

The refresh request must never inherit:

```
Authorization: Bearer <stale-access-token>
```

The refresh endpoint should depend on the HttpOnly cookie.

Use:

```
{
  url: "/auth/refresh",
  method: "POST"
}
```

Do not send:

```
body: {
  refreshToken
}
```

unless the backend contract explicitly requires it. Under the selected architecture, it should not.

5. REFRESH FUNCTION

Rewrite the refresh logic so it does NOT do this:

```
const refreshToken = api.getState().auth?.refreshToken;

if (!refreshToken) {
  throw unauthenticatedError();
}
```

That check is invalid for cookie-based refresh authentication.

Instead, simply call the refresh endpoint:

```
const refreshResult = await publicBaseQuery(
  {
    url: "/auth/refresh",
    method: "POST",
  },
  api,
  {
    ...extraOptions,
    skipAuthRefresh: true,
  }
);
```

Read the access token from the backend response.

Support the actual response shape already used by this application:

```
const refreshed = refreshResult.data?.data ?? refreshResult.data;
```

Require a valid access token:

```
if (
  refreshResult.error ||
  typeof refreshed?.accessToken !== "string" ||
  refreshed.accessToken.length === 0
) {
  throw unauthenticatedError();
}
```

Then update Redux:

```
api.dispatch(
  setAccessToken(refreshed.accessToken)
);
```

If the refresh response contains a user, update the user as well.

Do NOT construct or persist a frontend refresh token.

6. REFRESH TOKEN ROTATION

Assume the backend may rotate the refresh token.

The frontend must NOT attempt to manually manage that rotation.

The backend handles:

```
old HttpOnly refresh cookie
    ↓
validate
    ↓
rotate/revoke
```

↓

Set-Cookie: new HttpOnly refresh cookie

The browser handles the cookie automatically.

The frontend only cares about the returned access token.

Do not write code like:

```
refreshToken:  
  refreshed.refreshToken ?? oldRefreshToken
```

Remove that completely.

7. CONCURRENT 401 REQUESTS

Keep a module-scoped refresh promise so multiple simultaneous 401 responses share a single refresh operation.

Example behavior:

Request A → 401

Request B → 401

Request C → 401

↓

ONE /auth/refresh request

↓

all requests wait for same promise

↓

new access token

↓

A retries once


```
B retries once  
C retries once
```

This is especially important because refresh-token rotation can invalidate the previous refresh token.

Do not allow:

```
A → refresh  
B → refresh  
C → refresh
```

at the same time.

8. PREVENT REFRESH LOOPS

A refresh request must never recursively trigger another refresh.

Use:

```
skipAuthRefresh: true
```

for the refresh operation.

After a successful refresh, retry the original request exactly once.

Do NOT do:

```
401  
→ refresh  
→ 401  
→ refresh  
→ 401  
→ refresh
```

The retry must be final.

9. FINAL 401 AFTER REFRESH

Handle this case explicitly:

```
original request → 401  
refresh → successful  
retry → 401
```

The second 401 means the new access token did not authenticate the request.

Do not silently leave Redux looking authenticated.

Clear the local authentication session when appropriate.

For example:

```
clearLocalAuthSession(api.dispatch);
```

or dispatch the appropriate auth clear action.

Do not trigger another refresh.

The application should transition to an unauthenticated state.

10. REFRESH FAILURE

If:

```
request → 401  
refresh → 401/403/failure
```

then:

1. Clear Redux auth state.
2. Clear any frontend auth/session storage.
3. Reset user-specific RTK Query caches.
4. Return a normalized unauthenticated error.
5. Do not retry the original request again.
6. Do not enter a refresh loop.

The refresh cookie should be invalidated/cleared by the backend.

11. LOGOUT

Logout must happen in this order:

1. Keep current access token available
2. POST /auth/logout
3. Backend revokes session / clears refresh cookie
4. Reset RTK Query caches
5. Clear Redux auth state
6. Run local teardown callbacks

Do NOT clear Redux credentials before making the logout request if the logout endpoint requires the bearer access token.

The current approach of keeping credentials until the logout request is created is correct.

However, prevent logout from unnecessarily starting a refresh cycle.

Prefer making logout a no-refresh endpoint.

For example, include:

```
"logout"
```

in the endpoints that should not trigger automatic refresh.

If logout returns 401, treat server logout as best-effort and still perform local cleanup.

12. LOGOUT CACHE RESET

This is important.

After logout, reset user-specific RTK Query cache.

Use the appropriate API instance:

```
baseApi.util.resetApiState()
```

and reset other independent `createApi` instances if they exist.

For example, if:

```
profileApi  
authApi  
courseworkApi  
notificationApi
```

are separate API slices, determine which ones have user-specific cached data and reset them appropriately.

The goal is:

```
User A logs out  
↓  
User A cached data removed  
↓  
User B logs in  
↓  
User B cannot receive User A's cached API responses
```

Do not blindly reset unrelated global application state unless necessary.

13. PROFILE API

Inspect `profileApi`.

Make sure it uses the same authenticated `baseQueryWithRefresh` infrastructure.

If `profileApi` uses its own plain:

```
fetchBaseQuery(...)
```

then `/users/me` may bypass the refresh logic.

Fix this so authenticated profile requests participate in:

```
Bearer access token  
→ 401  
→ refresh via cookie  
→ retry once
```

14. GET CURRENT PROFILE

`getCurrentProfile()` should force a network request during session bootstrap.

The existing idea of:

```
forceRefetch: true
```

is good.

However, do not use `/users/me` as the only way to recover an access token after a page reload if the access token is intentionally memory-only.

On application startup:

```
Redux has no accessToken
  ↓
attempt POST /auth/refresh
  ↓
HttpOnly refresh cookie automatically sent
  ↓
new access token
  ↓
store access token in Redux
  ↓
fetch /users/me
  ↓
store user
```

This should be the primary bootstrap flow if the backend supports cookie-based refresh.

Do not expect `localStorage` to contain the refresh token.

15. AUTH PROVIDER / SESSION BOOTSTRAP

Inspect the `AuthProvider` or equivalent startup authentication logic.

It should handle these states clearly:

```
initializing
authenticated
unauthenticated
```

On application startup:

```
initializing
  ↓
try refresh
  ↓
success → set accessToken
  ↓
fetch current profile
  ↓
authenticated
```

If refresh fails:

```
initializing
  ↓
refresh fails
  ↓
clear auth
  ↓
unauthenticated
```

Avoid rendering protected application content while authentication state is still being determined.

Do not accidentally redirect to login before the refresh bootstrap finishes.

16. LOGIN

After successful login:

```
POST /auth/login
  ↓
backend sets HttpOnly refresh cookie
  ↓
response returns accessToken + user
  ↓
Redux setSession({
```

```
    accessToken,  
    user  
  })
```

Do not expect the frontend to receive/store a refresh token.

If the backend currently returns a refresh token in JSON, recommend changing the backend contract so it instead sets:

```
Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=...
```

Do not store the returned refresh token in frontend storage.

17. REGISTRATION

Apply the same rule to registration.

If registration authenticates the user:

```
register  
→ backend sets HttpOnly refresh cookie  
→ returns accessToken + user  
→ Redux stores accessToken + user
```

If registration does not authenticate the user, follow the backend's actual contract.

Do not introduce refresh-token persistence.

18. OAUTH

Inspect the OAuth flow.

OAuth redirects should ultimately establish the same session architecture:

```
OAuth provider  
  ↓  
backend callback  
  ↓  
backend establishes HttpOnly refresh cookie
```

```
↓  
frontend receives/obtains access token  
↓  
Redux stores accessToken + user
```

Do not create a second authentication architecture for OAuth.

Do not store OAuth refresh tokens in localStorage.

If the OAuth callback currently returns credentials in a URL, identify security problems such as tokens in query parameters/fragments and fix them where possible.

19. CSRF

Keep the CSRF mechanism if the backend uses a double-submit CSRF token.

Current approach:

```
CSRF cookie  
+  
x-csrf-token header
```

is acceptable if it matches the backend contract.

The CSRF cookie may be JavaScript-readable.

The refresh cookie must remain HttpOnly.

Do not confuse:

```
CSRF cookie
```

with:

```
refresh-token cookie
```

They have different purposes.

If possible, only attach the CSRF header to state-changing requests:


```
POST
PUT
PATCH
DELETE
```

rather than every GET request.

However, do not change this if the backend explicitly requires the CSRF header on every request.

20. COOKIE REQUIREMENTS

Document the backend expectations.

The refresh cookie should normally use appropriate settings such as:

```
HttpOnly
Secure
SameSite=Lax/Strict/None depending on deployment architecture
Path=/v1/auth
```

The exact `SameSite`, `Domain`, and `Path` values must match the frontend/backend deployment.

Do not invent cookie settings blindly.

If frontend and backend are on different sites, `SameSite=None; Secure` may be required.

Also verify backend CORS allows credentials:

```
Access-Control-Allow-Credentials: true
```

and does NOT use:

```
Access-Control-Allow-Origin: *
```

for credentialed requests.

21. ERROR NORMALIZATION

Keep the existing error normalization concept, but make sure it does not accidentally hide important authentication states.

At minimum distinguish:

```
401 → unauthenticated
403 → forbidden
5xx → service unavailable
other → request failed
```

Do not convert every authentication error into a generic server error.

Keep useful backend error information where safe.

22. shouldForceLogout

Review:

```
shouldForceLogout(endpoint, statusCode)
```

The current rule:

```
statusCode === 401
```

plus special `/users/me` 404 handling may be valid, but determine whether it is still needed after centralizing refresh/session handling.

Do not keep dead authentication logic merely because it existed previously.

If the function is used elsewhere, preserve its public behavior unless there is a strong reason to change it.

23. NORMALIZE AUTH RESPONSE

Review:

```
normalizeAuthResponse()
```

Make the frontend user object canonical.

Avoid unnecessary duplication such as:

```
avatar  
avatarUrl  
banner  
bannerUrl
```

unless the application genuinely requires both.

Do not include:

```
refreshToken
```

in the normalized auth response.

The normalized result should conceptually be:

```
{  
  accessToken,  
  user  
}
```

If the backend returns multiple response formats, keep compatibility intentionally, but do not let that create security problems.

24. ACCESS TOKEN PERSISTENCE

Prefer:

```
accessToken → Redux memory  
refreshToken → HttpOnly cookie
```

Do not persist the access token in localStorage unless there is a documented requirement.

On page reload:

```
Redux accessToken disappears
  ↓
AuthProvider starts bootstrap
  ↓
POST /auth/refresh
  ↓
browser sends HttpOnly cookie
  ↓
new accessToken
  ↓
Redux
```

This is expected behavior.

Do not try to make localStorage behave like a refresh-token store.

25. REFRESH / LOGOUT RACE CONDITION

There is a subtle race that must be handled.

Example:

```
Request A → 401
  ↓
refresh starts

User clicks Logout
  ↓
logout starts
  ↓
local auth is cleared

refresh finishes
  ↓
refresh code dispatches setAccessToken(...)
```

That could accidentally authenticate the user again after logout.

Prevent this.

Use a session generation/version/request ID or equivalent mechanism.

For example:

```
sessionVersion = 1

refresh starts with version 1

logout increments version to 2

refresh completes

refresh checks:
  current version === original version?

NO

Therefore:
  do not dispatch refreshed credentials
```

Choose the cleanest implementation for the existing architecture.

Do not introduce unnecessary complexity if there is already a reliable cancellation mechanism.

26. MULTIPLE TABS

Consider browser multi-tab behavior.

Because the access token is memory-only, each tab may have its own access token while sharing the HttpOnly refresh cookie.

At minimum ensure:

```
expired access token
→ refresh cookie
→ new access token
```

works independently in each tab.

Be careful with refresh-token rotation.

If multiple tabs simultaneously refresh and the backend uses strict one-time refresh-token rotation, coordinate refresh operations across tabs if required by the backend.

Do not implement cross-tab complexity unless the backend actually requires it, but explicitly identify the risk.

27. API ENDPOINT CLASSIFICATION

Review:

```
PUBLIC_AUTH_ENDPOINTS
PUBLIC_DISCOVERY_ENDPOINTS
```

Make the classification explicit.

For example:

```
const NO_REFRESH_ENDPOINTS = new Set([
  "login",
  "register",
  "refresh",
  "logout",
]);
```

Do not rely unnecessarily on endpoint names hidden inside unrelated logic.

Make sure public endpoints do not accidentally receive an Authorization header.

Authenticated endpoints should receive the Redux access token.

Refresh should use the cookie only.

28. EXPECTED FINAL ARCHITECTURE

The finished system should behave exactly like this:

Login

```
POST /auth/login
  ↓
Backend validates credentials
  ↓
```

Backend sets HttpOnly refresh cookie

↓

Response:
 accessToken
 user

↓

Redux:
 accessToken
 user

Normal API request

Redux accessToken

↓

Authorization: Bearer <token>

↓

API

Expired access token

API → 401

↓

POST /auth/refresh

↓

HttpOnly cookie automatically sent

↓

Backend validates/rotates refresh session

↓

new accessToken

↓

Redux updated

↓

original request retried once

Refresh failure

API → 401

↓

refresh fails

↓

clear auth

↓

reset user cache

↓
unauthenticated

Logout

POST /auth/logout
↓
backend revokes session
↓
backend clears refresh cookie
↓
reset RTK Query cache
↓
clear Redux auth

Page reload

Redux empty
↓
AuthProvider bootstrap
↓
POST /auth/refresh
↓
HttpOnly cookie
↓
new access token
↓
GET /users/me
↓
Redux user + accessToken

29. DO NOT BREAK THESE EXISTING FEATURES

While fixing authentication, preserve:

- RTK Query endpoint injection
- API tag types
- existing route structure
- `/v1` API normalization
- CSRF behavior compatible with backend
- existing login/register response compatibility

- OAuth functionality
- profile fetching
- profile updates
- logout callbacks
- existing UI routing
- existing protected-route behavior

Do not rewrite unrelated application code.

30. REQUIRED CODE REVIEW PROCESS

Before changing code:

1. Search the entire project for:

`refreshToken`

1. Search for:

`accessToken`

1. Search for:

`SESSION_KEY`

1. Search for:

`localStorage`
`sessionStorage`

1. Search for:

`/auth/refresh`

1. Search for:

`/auth/logout`

1. Search for:

getCurrentProfile

1. Find every `createApi`.
2. Find every `fetchBaseQuery`.
3. Find the AuthProvider/session bootstrap.
4. Find every place authentication state is cleared.
5. Find every place authentication state is initialized.

Then make the smallest coherent set of changes needed to implement the architecture above.

31. TEST ALL AUTH SCENARIOS

After making changes, verify these cases:

Test 1 — Login

```
login
→ refresh cookie exists
→ accessToken in Redux
→ user in Redux
```

Test 2 — Normal request

```
Authorization header contains access token
```

Test 3 — Access token expiration

```
request → 401
→ exactly one refresh
→ request succeeds after retry
```

Test 4 — Multiple simultaneous 401s

```
5 requests → 401
→ exactly one refresh request
→ all eligible requests retry once
```

Test 5 — Invalid refresh cookie

```
request → 401  
→ refresh fails  
→ auth cleared  
→ cache cleared  
→ no infinite loop
```

Test 6 — Logout

```
logout request has current bearer token  
→ local state cleared  
→ RTK Query cache reset  
→ refresh session revoked
```

Test 7 — Logout during refresh

```
refresh in progress  
→ logout  
→ refresh completion must NOT re-authenticate user
```

Test 8 — Page reload

```
reload  
→ Redux has no access token  
→ refresh cookie is used  
→ access token restored  
→ /users/me fetched
```

Test 9 — New user after logout

```
User A logout  
→ User B login  
→ User B must not see User A cached API data
```

Test 10 — OAuth

```
OAuth login  
→ same cookie-based refresh architecture  
→ no refresh token stored in localStorage
```

32. IMPORTANT: DO NOT ASSUME THE BACKEND RESPONSE

Before finalizing the frontend changes, inspect the existing backend/frontend API contract if available.

Determine exactly whether:

```
POST /auth/login
POST /auth/register
POST /auth/refresh
POST /auth/logout
```

return:

```
{
  accessToken,
  user
}
```

or:

```
{
  data: {
    accessToken,
    user
  }
}
```

or another shape.

Adapt the frontend to the actual contract instead of inventing a new response format.

Likewise verify the actual cookie name, CSRF cookie name, and OAuth behavior from the existing project configuration.

33. OUTPUT REQUIREMENTS

After implementing the fixes, provide:

1. A concise summary of what was changed.
2. A list of files changed.
3. Any backend requirements that must be satisfied.
4. Any remaining potential issues.
5. A short authentication flow diagram.
6. Tests/checks performed.
7. Explicitly confirm:

```
Refresh token is NOT stored in Redux.  
Refresh token is NOT stored in localStorage.  
Refresh token is NOT stored in sessionStorage.  
Refresh token is handled by HttpOnly cookie.  
Access token is stored in Redux.  
Refresh requests use credentials: "include".  
Refresh requests do not send a bearer access token.  
Concurrent 401s share one refresh request.  
A retried request can only retry once.  
Logout cannot be undone by an in-flight refresh.  
User-specific RTK Query caches are cleared on logout.
```

Do not just patch the obvious error. Review the complete authentication lifecycle and make all related files consistent with the cookie-based architecture.

If something in the current code contradicts this architecture, prioritize the architecture above and explain the contradiction before/after fixing it.